STA 3402 – Machine Learning & Neural Networks

Lecture 2 (Day 2) – Dimension Reduction: Principal Component Analysis (PCA)

කථිකාචාර්ය: Dr. Samir, සංඛ්‍යාන අධ්‍යයන අංශය, කොළඹ විශ්වවිද්‍යාලය

Lecture recording එක, whiteboard එක, R code එක සහ PCA slides භාවිතා කර සකස් කළ සම්පූර්ණ සටහන් (සරල සිංහලෙන්)

> **මේ සටහන කියවන විදිහ**
> තාක්ෂණික වචන (technical terms) exam එකේදී සහ R code එකේදී ඉංග්‍රීසියෙන්ම එන නිසා ඒවා ඉංග්‍රීසියෙන්ම තියලා තේරුම සිංහලෙන් දීලා තියෙනවා. රූපවල සහ slides වල labels ඉංග්‍රීසියෙන් තියෙන්නේ ඒ නිසා.

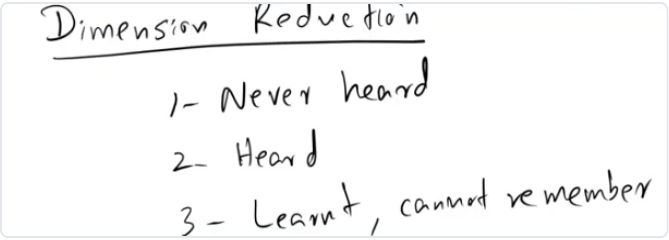
පටුන

1. පන්තියේ දැනුම්දීම් (assignment upload, නිවාඩු)
2. Dimension reduction කියන්නේ මොකක්ද?
3. PCA හි ප්‍රධාන අදහස
4. PCA හි පියවර
5. PCs කීයක් ගන්නද? – variance explained සහ scree (elbow) plot
6. R වල PCA – `prcomp()` සහ `princomp()`
7. R practical: mtcars මත PCA (පියවරෙන් පියවර, වළක්වා ගත යුතු වැරදි සමඟ)
8. Principal component එකක් ලියා දැක්වීම (loadings)
9. PCA යනු අතරමැදි පියවරක්
10. යෙදීමක්: image compression (පූසාගේ උදාහරණය)
11. PCA හි න්‍යාය – රූපමය පැහැදිලි කිරීම
12. Variables 3ක් හෝ වැඩි විට PCA, සහ PCA වැඩ නොකරන අවස්ථා
13. සාරාංශය, ගෙදර වැඩ සහ පුනරීක්ෂණ ප්‍රශ්න

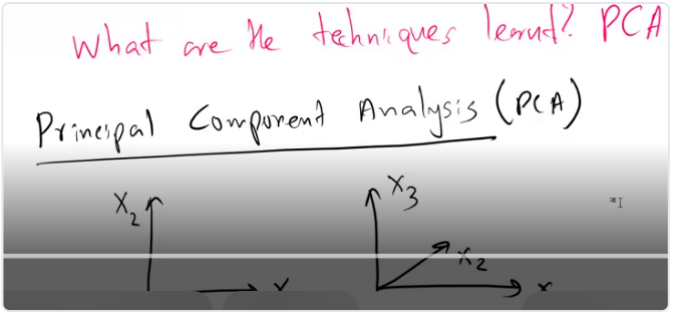
1. පන්තියේ දැනුම්දීම්

- ඔයාලගේ assignment files, දීපු link එකට deadline එකට කලින් upload කරන්න (lecturer "10.00 ට කලින්" කිව්වා – හරියටම වේලාව WhatsApp group එකෙන් බලන්න).
- **නත්තල් නිවාඩු:** ඊළඟ දවසේ හවස පටන් අරන් ලබන සඳුදා ඉවර වෙනවා. සඳුදා ඉඳන් lectures තියෙන නිසා ඊළඟ පන්තිය **29 වන සඳුදා**.
- ලබන සතියේ lab session එකට PCA ගැන **lab sheet** එකක් දෙනවා. Slides share කරනවා.

2. Dimension reduction කියන්නේ මොකක්ද?



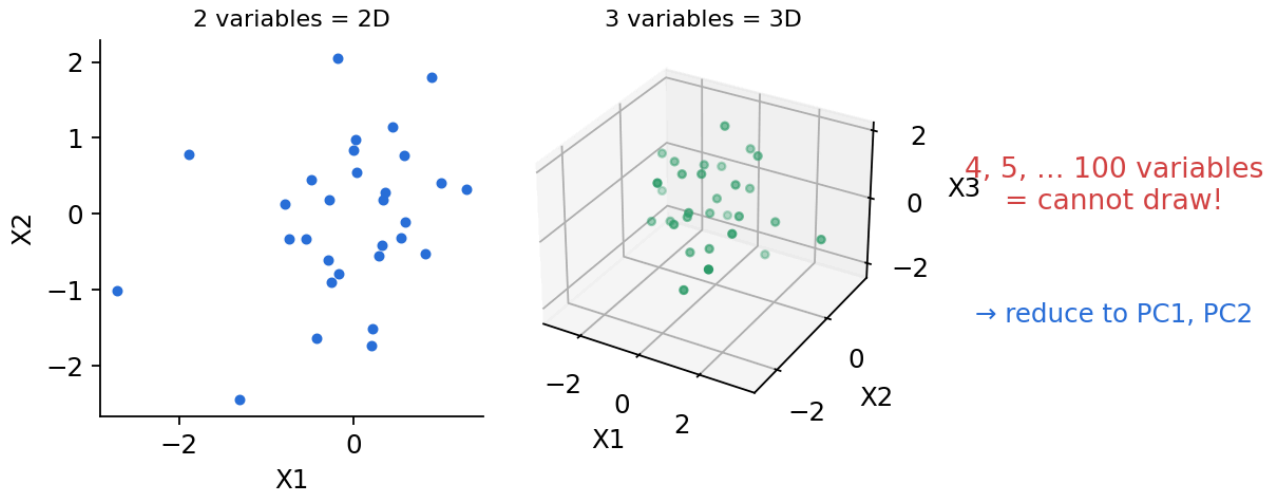
පන්තියේ සමීක්ෂණය: 1 = අහලාවත් නැහැ, 2 = අහලා තියෙනවා, 3 = ඉගෙනගත්තා, මතක නැහැ



ඉගෙනගත් ක්‍රමය: PCA. Variables 2 → 2D, 3 → 3D

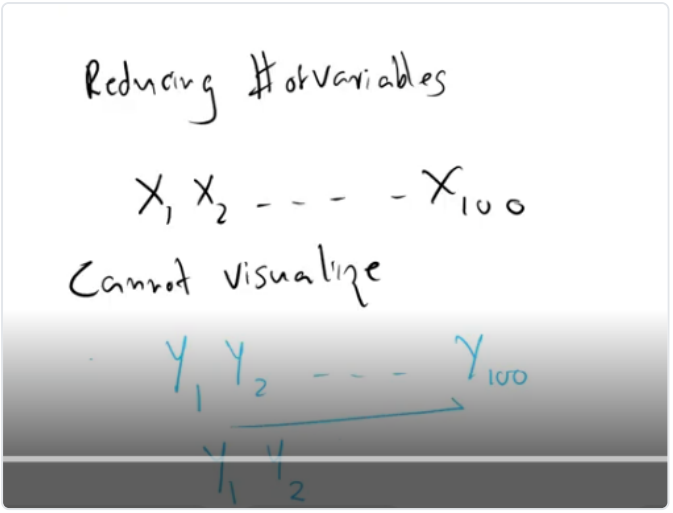
Dimension reduction ගැන අපි දන්නේ මොනවද කියලා lecturer ඇහුවා. සමහර ශිෂ්‍යයෝ optional multivariate statistics පාඨමාලාවකදී **Principal Component Analysis (PCA)** ඉගෙනගෙන තිබුණා. ඒ නිසා මෙතනදී PCA **මුල සිටම** උගන්වනවා.

- **Dimension = variables ගණන.** Variables දෙකක් (X1, X2) → 2-D graph එකක්. Variables තුනක් (X1, X2, X3) → 3-D graph එකක්. Variables වැඩි වෙන්න වෙන්න dimensions වැඩි වෙනවා.
- **Dimension reduction** = variables ගණන අඩු කිරීම.
- වැදගත්: මේකෙන් අදහස් කරන්නේ variables සමහරක් සරලව විසි කරනවා කියන එක **නෙවෙයි**. ගණනය කිරීමේදී ඔක්කොම variables සලකා බලනවා, නමුත් තොරතුරු වැඩි ප්‍රමාණයක් අඩු variables ගණනකට ගෙනෙන **අලුත් variables පද්ධතියක් හදනවා**.

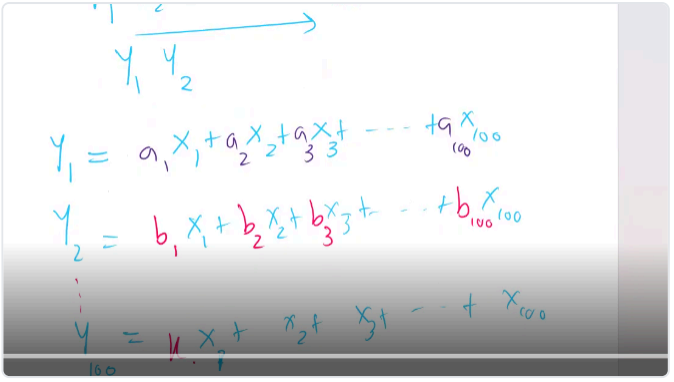


Dimensions 2ක් හෝ 3ක් අඳින්න පුළුවන්, නමුත් 100ක් බැහැ

3. PCA හි ප්‍රධාන අදහස



Whiteboard: variables ගණන අඩු කිරීම



Whiteboard: හැම අලුත් variable එකක්ම linear combination එකක්

Variables 100ක් $X_1, X_2, \dots, X_{100}$ තියෙනවා කියමු. ඒවා **visualize කරන්න බැහැ** (අපිට dimensions 3ක් දක්වා විතරයි පේන්නේ; 2 තමයි හොඳම).

ඒ නිසා **අලුත් variables 100ක** $Y_1, Y_2, \dots, Y_{100}$ කට්ටලයක් හදනවා, මෙහෙම:

- **Y_1** data වල **වැඩිම** විචලනය (variation) පැහැදිලි කරනවා,
- **Y_2 දෙවනියට වැඩිම** විචලනය පැහැදිලි කරනවා, මේ විදිහට **අවරෝහණ පිළිවෙළට (descending order)**.

හැම අලුත් variable එකක්ම **ඔක්කොම** මුල් variables වල **linear combination (රේඛීය සංයෝජනයක්)**:

$$Y_1 = a_1X_1 + a_2X_2 + a_3X_3 + \dots + a_{100}X_{100}$$

$$Y_2 = b_1X_1 + b_2X_2 + b_3X_3 + \dots + b_{100}X_{100}$$

$$\vdots$$

$$Y_{100} = k_1X_1 + k_2X_2 + \dots + k_{100}X_{100}$$

(a, b, ..., k නියත අගයන්.) මේ අලුත් variables වලට **principal components (PCs)** කියනවා. මුල් variables 100කින් PCs 100ක් ලැබෙනවා.

Y_1 සහ Y_2 තුළ විචලනයෙන් වැඩි කොටසක් තියෙන නිසා **Y_1 සහ Y_2 විතරක් plot කරන්න** පුළුවන්. දැන් data බලන්න පුළුවන්, සමහරවිට **clusters** හොයාගන්න පුළුවන්, ඊට පස්සේ උදාහරණයක් විදිහට **හැම cluster එකකටම වෙනම regression එකක්** fit කරන්න පුළුවන්. Variables (genes) දහස් ගණනක් තියෙන **bioinformatics** වගේ ක්ෂේත්‍රවල මේක ගොඩක් ප්‍රයෝජනවත්.

4. PCA හි පියවර

පන්තියේ ක්‍රියාකාරකම: "PCA හි ප්‍රධාන පියවර දෙන්න". ශිෂ්‍යයන්ගේ උත්තර: standardize කිරීම, covariance matrix එක ගැනීම, eigenvalues සහ eigenvectors ගණනය කිරීම, eigenvalues තෝරා ගැනීම, feature vector එක සෑදීම, transform කිරීම. Lecturer ගේ සම්පූර්ණ ලැයිස්තුව:

නීතිය 0 – data වර්ගය

PCA වැඩ කරන්නේ **quantitative (සංඛ්‍යාත්මක) data වලට විතරයි**. Categorical data වලට වැඩ කරන්නේ නැහැ. මුලින්ම categorical variables ඉවත් කරන්න.

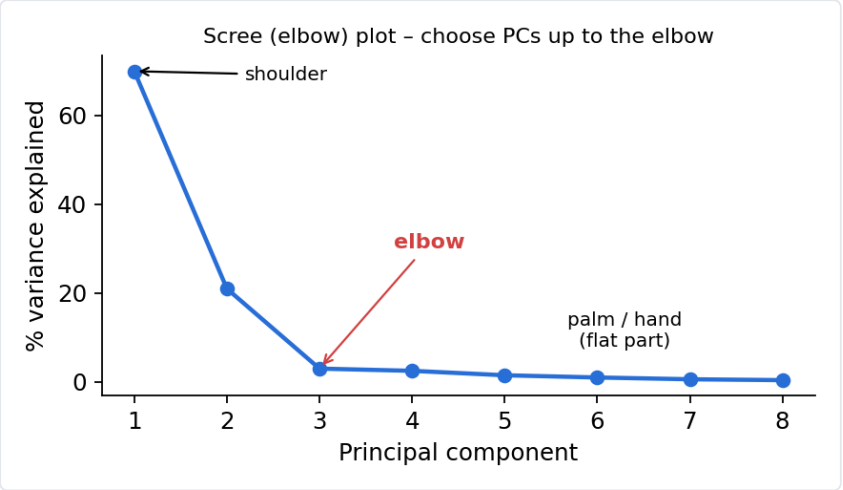
1. Data **standardize** කරන්න. Variables වෙනස් ඒකක වලින්, වෙනස් විචලනයන් සහිතව තියෙන්න පුළුවන් නිසා ඒවා එකම පරිමාණයකට ගේනවා (mean එක අඩු කරලා standard deviation එකෙන් බෙදනවා).
2. Standardize කළ data වල **variance–covariance matrix** එක සොයන්න.
3. මේ matrix එකේ **eigenvalues** සහ **eigenvectors** සොයන්න.
4. පිළිවෙළට සකසන්න: **විශාලතම eigenvalue** සහ එහි eigenvector → PC1; ඊළඟ විශාලතම → PC2; ආදී වශයෙන්.
5. Eigenvectors වලින් **loadings** ලැබෙනවා (a_1, a_2, … coefficients).
6. **අලුත් (transformed) data set එක** ලියන්න – PC scores.
7. Variance explained සහ **scree plot** එක භාවිතා කරලා **PCs කීයක්** ඕනද කියලා තීරණය කරන්න.

5. PCs කීයක් ගන්නද? – variance explained සහ scree plot

Variables 100ක් තියෙනවා නම් PCs 100ක් තියෙනවා. හැම PC එකකටම **පැහැදිලි කරන විචලනය (variation explained)** ගණනය කරන්න පුළුවන්. උදාහරණය:

PC	පැහැදිලි කරන විචලනය %	සමුච්චිත (cumulative) %
PC1	70%	70%
PC2	21%	70 + 21 = 91%
PC3	3%	94%
…	…	…

PCs 100ම තියාගෙන වැඩක් නැහැ – අපේ අරමුණ variables ගණන **අඩු කිරීම**. කීයක් තියාගන්නද කියලා තීරණය කරන්න **scree plot** එක (**elbow plot** කියලත් කියනවා) භාවිතා කරනවා: හැම PC එකකටම පැහැදිලි කරන variance එක plot කරනවා.



රේඛාව **අතක්** විදිහට හිතන්න: මුලින් තියෙන උස කොටස **උරහිස (shoulder)**, අගට තියෙන පැතලි කොටස **අත්ල (palm)**, නැමෙන තැන **වැලමිට (elbow)**. න්‍යාය කියන්නේ: **elbow point එක දක්වා PCs තියාගන්න**. මෙතන elbow එක PC2 හෝ PC3 → PCs 2ක් හෝ 3ක් ගන්න.

6. R වල PCA – prcomp() සහ princomp()

පන්තියේ උත්තරය: functions දෙකක් භාවිතා කරන්න පුළුවන්.

	`prcomp()`	`princomp()`
Package	**stats** (default, install/load කරන්න ඕන නැහැ)	**stats** (default, install/load කරන්න ඕන නැහැ)
ක්‍රමය	Data වල Singular value decomposition (SVD) – සංඛ්‍යාත්මකව වඩා නිවැරදියි	Covariance (හෝ correlation) matrix එකේ eigen decomposition
Loadings තියෙන තැන	`$rotation`	`$loadings`
Scores තියෙන තැන	`$x`	`$scores`
Standardize option	`center = TRUE, scale. = TRUE`	`cor = TRUE`
සටහන	පේළි වලට වඩා variables වැඩි වුණත් වැඩ කරනවා. සාමාන්‍යයෙන් වඩා කැමති මේකට.	Variables වලට වඩා පේළි වැඩි විය යුතුයි. $n - 1$ වෙනුවට n න් බෙදනවා.

Help page එකෙන් package එක බලන්න පුළුවන්: `?prcomp` එකේ "prcomp {stats}", `?princomp` එකේ "princomp {stats}" කියලා පෙන්නනවා.

7. R practical: mtcars මත PCA

ක්‍රියාකාරකම: (1) mtcars load කරන්න; (2) PCA කරන්න; (3) scree plot එක ගන්න `plot()` භාවිතා කරන්න; (4) ඔයා නිර්දේශ කරන්නේ PCs කීයක්ද?

Lecturer දුන්න පොදු රටාව: ඔයාගේ data වලට `data` කියලා නම නම්, `PCAout = prcomp(data)`, ඊට පස්සේ `plot(PCAout)` එකෙන් scree plot එක ලැබෙනවා.

පියවර 1 – Data load කිරීම

```
# STA 3402 – Day 2
# 1. load the mtcars dataset
data(mtcars)
dim(mtcars)          # පේළි 32, columns 11

# 2. carry out the PCA
prcomp()
princomp()
# do we need any additional packages?
?prcomp              # prcomp {stats} -> stats package එකේ එනවා
?princomp            # princomp {stats}
```

පළමු උත්සාහය – සහ එය වැරදි ඇයි

```
PCAout = prcomp(mtcars)
summary(PCAout)      # PC1 ≈ 92.7%, PC2 ≈ 7.2%, ඉතිරි ඒවා ඉතා කුඩායි
plot(PCAout)         # elbow එක PC 1 හෝ 2
```

මේකෙන් PC1 ≈ 92% සහ PC2 ≈ 7% ලැබෙනවා, හොඳ වගේ පේනවා, නමුත් **උත්තරය වැරදියි**. ඇයි? (ශිෂ්‍යයෙක් නිවැරදිව උත්තර දුන්නා.)

1. **Categorical variables ඉවත් කළේ නැහැ**. PCA වැඩ කරන්නේ quantitative data වලට විතරයි.
2. Data **standardize කළේ නැහැ**. `disp` සහ `hp` වල අංක ගොඩක් ලොකුයි (සිය ගණන්), ඒ නිසා ඒවායේ ඒකක නිසාම PC1 ඒවාගෙන් ආධිපත්‍යය ගන්නවා.

පියවර 2 – Categorical variables හොයාගැනීම

```
# remove the categorical variable
str(mtcars)          # හැම column එකක්ම "num" – මෙතන උදව්වක් නැහැ!
?mtcars              # variable විස්තරය කියවන්න
```

Data set එක හැමදේම අංක විදිහට store කරලා තියෙන නිසා `str()` හැම column එකක්ම numeric කියලා පෙන්නනවා. නමුත් ඒවායින් සමහරක් ඇත්තටම **categorical (qualitative)**. Help page එකට අනුව:

Variable	තේරුම	වර්ගය
mpg	ගැලුමකට සැතපුම්	Quantitative ✔
cyl	Cylinders ගණන	Quantitative (integer) ✔
disp	Displacement (engine ප්‍රමාණය)	Quantitative ✔
hp	Gross horsepower	Quantitative ✔
drat	Rear axle ratio	Quantitative ✔
wt	බර	Quantitative ✔
qsec	සැතපුම් කාලක් යන්න යන කාලය	Quantitative ✔
vs	Engine වර්ගය (0 = V-හැඩය, 1 = straight)	**Categorical ✘**
am	Transmission (0 = automatic, 1 = manual)	**Categorical ✘**
gear	ඉදිරි gears ගණන	Quantitative ලෙස ගත හැකියි ✔
carb	Carburetors ගණන	Quantitative ලෙස ගත හැකියි ✔

ඒ නිසා **vs** සහ **am** (columns 8 සහ 9) ඉවත් කරන්න ඕන.

පියවර 3 – vs සහ am ඉවත් කිරීම (copy එකක් මත වැඩ කරන්න!)

```
## create a copy of the dataset
data = mtcars

# option 1: vs සහ am ඉවත් කිරීම (නිර්දේශ කරන්නේ නැහැ)
data$vs = NULL
data$am = NULL

# option 2 (නිර්දේශිතයි): quantitative columns විතරක් ගැනීම
data_quants = data[,-c(8,9)]
data_quants = data[,c(1:7,10,11)]
dim(data_quants)     # පේළි 32, columns 9
```

- Built-in data set එකක **හැමවිටම copy එකක් හදාගන්න** (`data = mtcars`). මුල් එක වෙනස් කරන්න එපා, නැත්නම් ඊළඟ පාර සම්පූර්ණ data set එක ලැබෙන්නේ නැහැ.
- `data$vs = NULL` ඒ data frame එකෙන් column එක **ස්ථිරවම** මකනවා – **නිර්දේශ කරන්නේ නැහැ**.
- `data[ , -c(8,9)]` කියන්නේ: ඔක්කොම පේළි (row කොටස හිස්), 8 සහ 9 **හැර** ඔක්කොම columns. `data[ , c(1:7,10,11)]` columns 1–7, 10 සහ 11 තෝරනවා. දෙකෙන්ම එකම ප්‍රතිඵලය. මේක **නිර්දේශිතයි**, මොකද මුල් data වෙනස් වෙන්නේ නැහැ, පස්සේ analysis එකේදී categorical variables ඕන වෙන්නත් පුළුවන්.

> **Code එක run කරද්දී පරිස්සමෙන්**
> R file එකේ options දෙකම එකකට පස්සේ එකක් ලියලා තියෙනවා. ඔයා මුලින් option 1 run කළොත් `data` වල දැනටමත් columns 9යි, එතකොට `data[,-c(8,9)]` වැරදිලා **gear සහ carb** ඉවත් කරනවා. **එක option එකක් විතරක්** පාවිච්චි කරන්න: අලුත් copy එකක් (`data = mtcars`) මත option 2 run කරන්න.

වැරදි අදහසක්: sapply(data, is.numeric)

```
# this is not going to work
dim(data)
Numaric_data = data[, sapply(data, is.numeric)]
dim(Numaric_data)    # කිසිවක් ඉවත් වෙලා නැහැ!
```

- `sapply(data, is.numeric)` හැම column එකකටම `is.numeric()` යොදලා TRUE/FALSE දෙනවා. TRUE columns විතරක් තියාගන්නවා.
- නමුත් **vs සහ am අංක (0 සහ 1) විදිහට store කරලා** තියෙන නිසා ඒවාටත් `is.numeric()` TRUE දෙනවා, ඒ නිසා **කිසිවක් ඉවත් වෙන්නේ නැහැ**.
- Categorical variables නිසි ලෙස **factors** විදිහට store කරලා තිබුණොත් විතරයි මේක වැඩ කරන්නේ.
- පාඩම: AI tools සමහර වෙලාවට data එකේ ඇත්තටම තියෙන්නේ මොනවද කියලා නොදැන මේ වගේ code යෝජනා කරනවා. **හිතලා check කරන්න** (උදා: `dim()` එකෙන්).

පියවර 4 – Data standardize කිරීම

```
# standardize the data
data_quants$mpg
scale(data_quants$mpg)          # එක variable එකක්

data_standadized = scale(data_quants)   # ඔක්කොම columns එකවර
```

- `scale(x, center = TRUE, scale = TRUE)` mean එක අඩු කරලා (center) standard deviation එකෙන් බෙදනවා (scale). අඩු කිරීම විතරක් හෝ බෙදීම විතරක් ඕන නම් එකක් FALSE කරන්න.
- උදාහරණයක් විදිහට, mpg 21.0 (පළමු කාර් එක) 0.1509 බවට පත් වෙනවා.
- `scale(data_quants)` හැම column එකකටම මේක කරනවා. භාවිතා කළ means සහ standard deviations output එකේ (attributes විදිහට) ගබඩා වෙනවා.

තවත් ක්‍රමයක් (ශිෂ්‍යයෙකුගේ උත්තරය): `prcomp()` function එක ඇතුළේම standardize කරන්න දෙන්න:

```
pca_result = prcomp(data_quants, center = TRUE, scale. = TRUE)
```

මේකෙන් `prcomp(scale(data_quants))` එකේ ප්‍රතිඵලයමයි ලැබෙන්නේ.

පියවර 5 – PCA run කරලා summary එක කියවීම

```
prcomp_out = prcomp(data_standadized)
summary(prcomp_out)
```

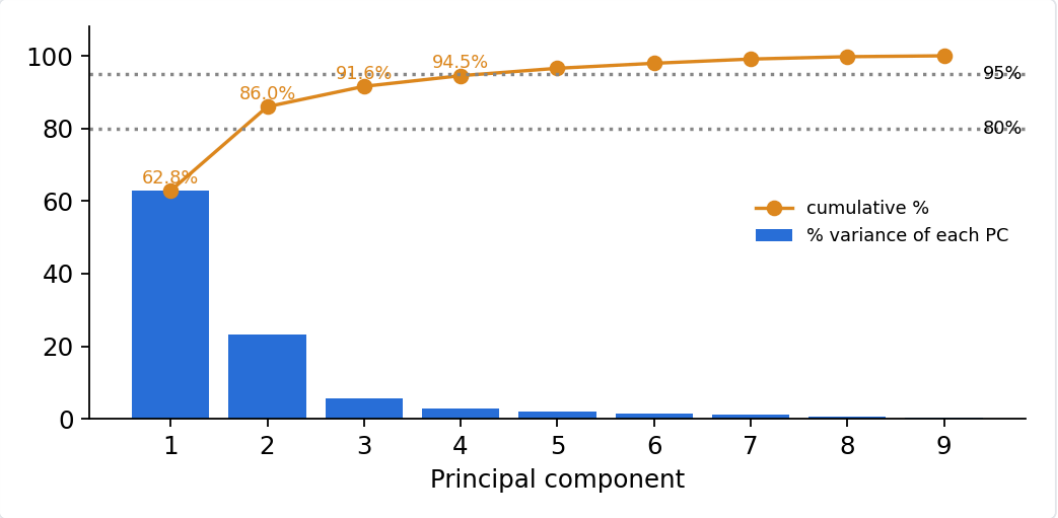
පන්තියේදී lecturer ගේ වැරැද්ද (සහ එය වැදගත් වන්නේ ඇයි)
පන්තියේදී lecturer මුලින් standardize කළේ `data_quants` වෙනුවට (තවමත් vs සහ am තිබුණු) `data`. ප්‍රතිඵලය PC1 = 60.08%, PC2 = 24.1%, PC3 = 5.7%, PC4 = 2.45% (cumulative 60.08%, 84.17%, 89.87%, 92.32%). පස්සේ loadings බලද්දී output එකේ **vs සහ am** තියෙනවා දැකලා වැරැද්ද තේරුම් ගත්තා – ඒවා එතන තියෙන්න බැහැ. පාඩම: **අවධානයෙන් ඉන්න, ලැබෙන ඕනම output එකක් එහෙම්මම පිළිගන්න එපා**. ඒක තේරුමක් තියෙනවද කියලා හිතන්න.

නිවැරදි output එක (standardize කළ, quantitative variables 9):

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9
Standard deviation	2.378	1.443	0.710	0.515	0.428	0.352	0.324	0.242	0.149
Proportion of variance	0.628	0.231	0.056	0.029	0.020	0.014	0.012	0.007	0.003
Cumulative proportion	0.628	0.860	**0.916**	0.945	0.966	0.979	0.991	0.998	1.000

කියවන විදිහ: PC1 විචලනයෙන් 62.8% පැහැදිලි කරනවා. PC2 23.1%. PC1 + PC2 එකට 86.0% (**cumulative** අගය ඒවා එකතු කරනවා). PC1 + PC2 + PC3 එකට **91.58%**.

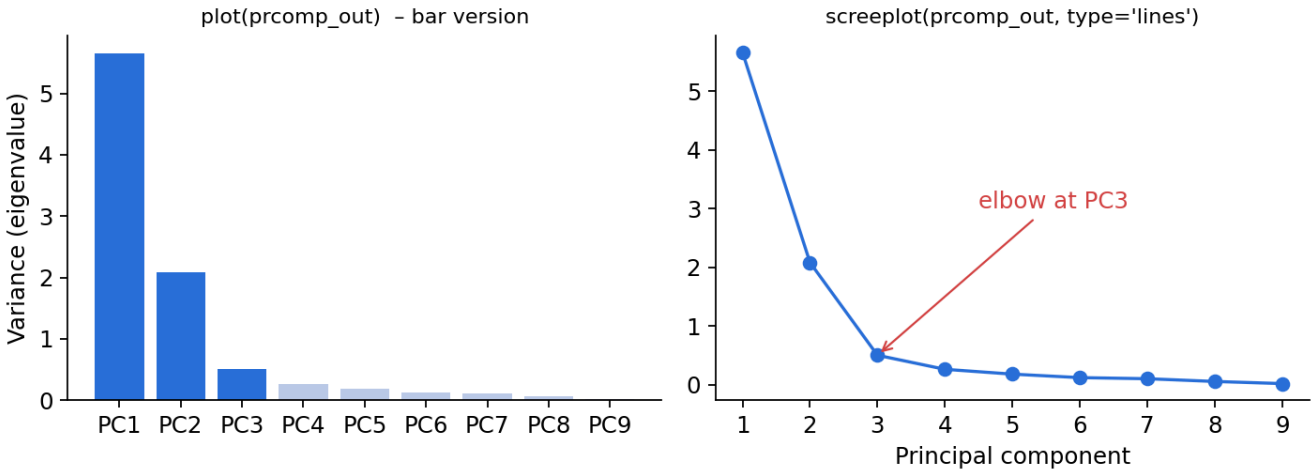
Proportion of variance = (ඒ PC එකේ sd)² ÷ ඔක්කොම sd² වල එකතුව. Data standardize කරලා තියෙන නිසා මුළු variance = variables ගණන = 9. PC1 සඳහා: $2.378^2 = 5.66$, $5.66 / 9 = 0.628$.



හැම PC එකකම variance (bars) සහ cumulative % (රේඛාව)

පියවර 6 – Scree plot සහ PCs කීයක්ද

```
#3. scree plot
plot(prcomp_out)          # variances වල bar chart එක
screeplot(prcomp_out)     # එකම plot එක
# screeplot(prcomp_out, type = "lines") -> රේඛා version එක
```



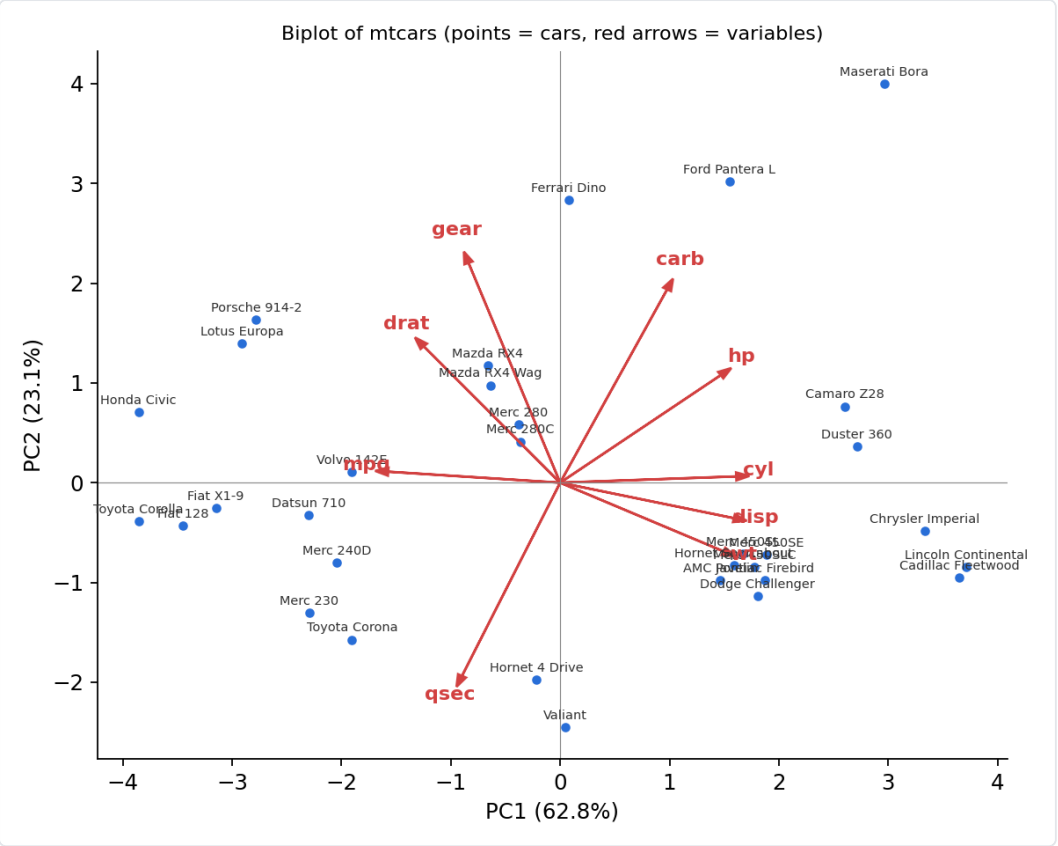
PCs 1, 2, 3 හෝ 4 න් නවතින්නද? අංක වලින් විතරක් තීරණය කරන එක ලේසි නැහැ, ඒ නිසා scree plot එක බලනවා. PC3 න් පස්සේ plot එක **x-අක්ෂයට බොහෝ දුරට සමාන්තරයි** (පැතලියි). ඒ නිසා **elbow point එක 3**.

> **4 වන ප්‍රශ්නයට උත්තරය**
>
> **PCs 3ක්**, මොකද (a) scree plot එකේ elbow එක 3 හි තියෙනවා, සහ (b) පළමු PCs 3 data විචලනයෙන් **91.58%** ක් පැහැදිලි කරනවා, ඒක හොඳ ප්‍රතිඵලයක්.
> "ප්‍රමාණවත්" කියන්නේ කොච්චරද කියන එක ක්ෂේත්‍රය සහ අවශ්‍ය නිරවද්‍යතාව මත රඳා පවතිනවා: සමහර යෙදුම් වලට 95% ට වැඩි ඕන, සමහර ඒවාට 80% ඇති.

පියවර 7 – Biplot එකකින් visualize කිරීම

```
# visualize
biplot(prcomp_out)
```

`biplot()` **පළමු PCs දෙක** plot කරනවා: හැම කාර් එකක්ම point එකක් (x-අක්ෂයේ PC1, y-අක්ෂයේ PC2), හැම මුල් variable එකක්ම රතු ඊතලයක්. Clusters දකින්න පුළුවන්, සහ අමතර විස්තර: එකම දිශාවට යොමු වෙන variables එකිනෙකට සම්බන්ධයි.



බර, බලවත් කාර් (Cadillac, Lincoln) දකුණු පැත්තේ; කුඩා, ඉන්ධන ඉතිරි කරන කාර් (Toyota Corolla, Honda Civic) වම් පැත්තේ. ඔයාගේ R plot එක කැඩපතක ප්‍රතිබිම්බයක් වගේ පෙරළිලා පේන්න පුළුවන් – ඒක කමක් නැහැ (8 කොටසේ සටහන බලන්න).

8. Principal component එකක් ලියා දැක්වීම

PC එකක් කියන්නේ මුල් (standardize කළ) variables වල linear combination එකක්:

$$\text{PC1} = a_1 \cdot \text{mpg} + a_2 \cdot \text{cyl} + a_3 \cdot \text{disp} + a_4 \cdot \text{hp} + a_5 \cdot \text{drat} + a_6 \cdot \text{wt} + a_7 \cdot \text{qsec} + a_8 \cdot \text{gear} + a_9 \cdot \text{carb}$$

a_1, a_2, … ගන්නේ කොහොමද? Output object එක ඇතුළේ තියෙන දේවල් බලන්න:

```
str(prcomp_out)      # කොටස් පෙන්නනවා: sdev, rotation, center, scale, x
?prcomp              # "Value" කොටසේ හැම එකක්ම පැහැදිලි කරනවා
```

කොටස	ඇතුළේ තියෙන දේ
`sdev`	PCs වල standard deviations (eigenvalues වල වර්ගමූලය)
`rotation`	Variable **loadings** matrix එක – එහි columns තමයි **eigenvectors**. (`princomp` මේකට `loadings` කියනවා.)
`center`, `scale`	භාවිතා කළ centering සහ scaling
`x`	**Transform කළ data** (PC scores)

```
# initial data
data_quants          # columns: mpg, cyl, disp, ...
# transformed data
prcomp_out$x         # columns: PC1, PC2, PC3, ...
# loadings
prcomp_out$rotation
```

මුල් data වල පළමු variable එක mpg, දෙවැන්න cyl. Transform කළ data වල පළමු variable එක **PC1**, දෙවැන්න **PC2**. හැම PC එකක්ම ඔක්කොම මුල් variables වල මිශ්‍රණයක්.

> **සුප් (වට්ටෝරුව) උපමාව**
> PC එකක් සුප් වට්ටෝරුවක් වගේ: "මෙච්චර කැරට් + මෙච්චර ලීක්ස් + මෙච්චර ප්‍රෝටීන් + මෙච්චර ලුණු + මෙච්චර කුළුබඩු...". Loadings වලින් කියන්නේ හැම variable එකකින්ම PC එකට කොච්චර එකතු වෙනවද කියලා.

Loadings (නිවැරදි, variables 9)

Variable	PC1	PC2	PC3
mpg	−0.3931	0.0275	−0.2212
cyl	0.4026	0.0157	−0.2523
disp	0.3974	−0.0889	−0.0783
hp	0.3671	0.2694	−0.0172
drat	−0.3118	0.3417	0.1500
wt	0.3735	−0.1719	0.4537
qsec	−0.2244	−0.4840	0.6281
gear	−0.2095	0.5508	0.2066
carb	0.2446	0.4843	0.4641

PC1 (R code එකෙන්):

$$\text{PC1} = -0.3931\ \text{mpg} + 0.4026\ \text{cyl} + 0.3974\ \text{disp} + 0.3671\ \text{hp} - 0.3118\ \text{drat} + 0.3735\ \text{wt} - 0.2244\ \text{qsec} - 0.2095\ \text{gear} + 0.2446\ \text{carb}$$

තේරුම: cylinders වැඩි, ලොකු engine, horsepower වැඩි, බර වැඩි කාර් වලට PC1 විශාලයි; mpg වැඩි කාර් වලට PC1 කුඩායි. ඒ නිසා PC1 **"ලොකු, බලවත්, ඉන්ධන වැඩිපුර කන කාර්"** score එකක් වගේ.

ගෙදර වැඩ උත්තරය: PC2 සහ PC3

$$PC2 = 0.0275\ mpg + 0.0157\ cyl - 0.0889\ disp + 0.2694\ hp + 0.3417\ drat - 0.1719\ wt - 0.4840\ qsec + 0.5508\ gear + 0.4843\ carb$$

$$PC3 = -0.2212\ mpg - 0.2523\ cyl - 0.0783\ disp - 0.0172\ hp + 0.1500\ drat + 0.4537\ wt + 0.6281\ qsec + 0.2066\ gear + 0.4641\ carb$$

> **ලකුණු (signs) ගැන සටහන**
> R එකේ PC එකක ඔක්කොම loadings වලට **ප්‍රතිවිරුද්ධ ලකුණ** පෙන්නන්න පුළුවන් (උදා: හැම PC2 අගයක්ම −1 න් ගුණ කරලා). ඒක කමක් නැහැ: "ඉස්සරහට" හෝ "පස්සට" යොමු වුණත් එකම රේඛාවයි. හැමවිටම ඔයාගේම R output එකේ අංක පාවිච්චි කරන්න.

PC score එකක් අතින් ගණනය කිරීම (පළමු කාර් එක, Mazda RX4)

```
data_standadized[1,]    # dataset එකේ පළමු record එක (standardize කළ)
prcomp_out$x[1,1]       # පළමු කාර් එකේ PC1 score එක
```

Mazda RX4 හි standardize කළ අගයන්: mpg 0.1509, cyl −0.1050, disp −0.5706, hp −0.5351, drat 0.5675, wt −0.6104, qsec −0.7772, gear 0.4236, carb 0.7352.

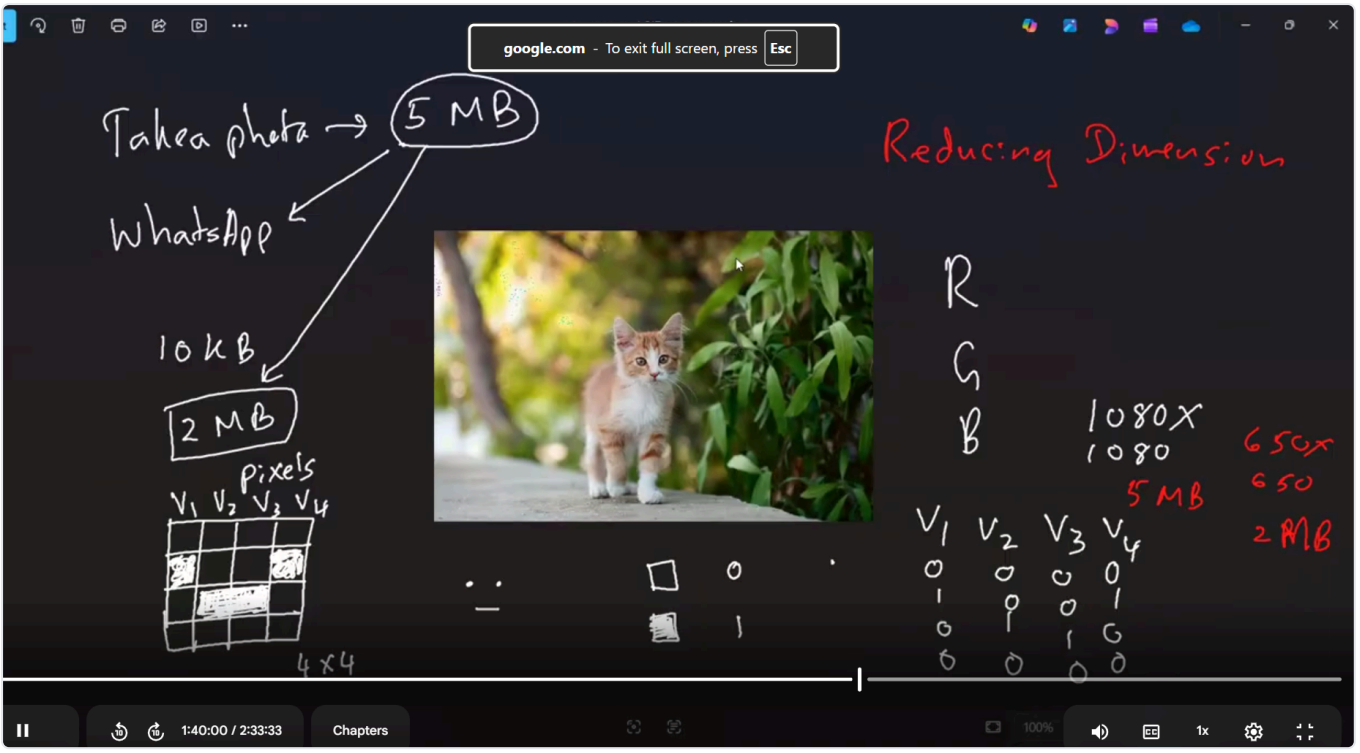
Variable	Loading × standardize කළ අගය	ප්‍රතිඵලය
mpg	−0.3931 × 0.1509	−0.0593
cyl	0.4026 × −0.1050	−0.0423
disp	0.3974 × −0.5706	−0.2267
hp	0.3671 × −0.5351	−0.1964
drat	−0.3118 × 0.5675	−0.1770
wt	0.3735 × −0.6104	−0.2280
qsec	−0.2244 × −0.7772	+0.1744
gear	−0.2095 × 0.4236	−0.0887
carb	0.2446 × 0.7352	+0.1798
එකතුව = PC1 score එක		**−0.664**

මේක `prcomp_out$x[1,1]` එකට ගැලපෙනවා. ඒ කියන්නේ transform කළ data කියන්නේ "loadings × standardize කළ data" විතරයි.

9. PCA යනු අතරමැදි පියවරක්

> වැඩ කරන්න අමාරු variables ගොඩක් තියෙනකොට අපි ඒවා PCs කිහිපයකට අඩු කරනවා. ඊට පස්සේ **තෝරාගත් PCs ඊළඟ පියවර** සඳහා භාවිතා කරනවා, උදා: **regression** හෝ **clustering**.

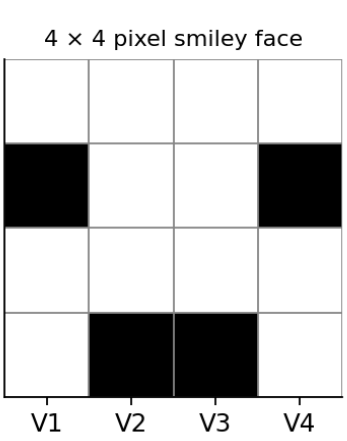
10. යෙදීමක්: image compression (පූසාගේ උදාහරණය)



Whiteboard: photo size, pixels, RGB channels සහ dimension අඩු කිරීම

10.1 අදහස

- ඔයා phone එකෙන් photo එකක් ගන්නවා: **5 MB**. ඒක **WhatsApp** හරහා යවනවා. යාළුවා download කරද්දී ඒක සමහරවිට **2 MB** හෝ **10 KB** විතරයි. Messaging apps වල compression එක ගොඩක් හොඳයි; ඉඩ ඉතිරි කරගන්න සමහරු තමන්ටම WhatsApp එකෙන් photos/videos යවනවා.
- Image එකක් හැදිලා තියෙන්නේ **pixels** වලින්. කළු-සුදු image එකක් **4 × 4 resolution** එකක් කියමු. හැම pixel එකක්ම හිස් (0) හෝ පාට කරපු (1). ඒ නිසා smiley face එකක් අංක විදිහට store කරන්න පුළුවන්. හැම column එකක්ම variable එකක් V1, V2, V3, V4.



Stored as numbers
(1 = coloured, 0 = empty)

V1	V2	V3	V4
0	0	0	0
1	0	0	1
0	0	0	0
0	1	1	0

- පාට image එකක **channels තුනක්: Red, Green, Blue (RGB)**. හැම channel එකක්ම අංක matrix එකක්.
- **1080 × 1080** pixels වගේ resolution එකක් (උදා: 5 MB). ඒක **650 × 650** කට compress කළාම සමහරවිට 2 MB. Compression = **dimensions අඩු කිරීම** = ඉඩ අඩු කිරීම, **image එකට හානි නොකර** (වටිනා තොරතුරු නැති වෙන්න බැහැ).
- මේක PCA හි සෘජු යෙදීමක්: image එකේ පළමු PCs කිහිපය විතරක් තියාගන්නවා.

10.2 ප්‍රතිඵලය: PCs වැඩි වෙන්න වෙන්න image එක පැහැදිලියි



හැම පාට channel එකකම පළමු k PCs විතරක් භාවිතා කරලා පූසාව නැවත හැදීම (lecture image එකෙන් හැදුවේ)

PCs ඉතා ටිකක් තියෙනකොට පූසෙක් පේන්නේ නැහැ. PCs වැඩි කරද්දී (ඊළඟ 10%, 20%, 30% ...) image එක පැහැදිලි වෙනවා. PCs 30ක් පමණ වෙද්දී පූසා පැහැදිලියි නමුත් ටිකක් noise (ඇට ඇට වගේ කොටස්) තියෙනවා; PCs ගොඩක් තියෙනකොට මුල් එක වගේම පේනවා. ඉතින් අපි variables අඩු කරනවා, නමුත් **තොරතුරු තියාගන්නවා**: variation explained එකට රේඛාවක් අඳිනවා, ඊට වැඩි PCs තියාගන්නවා.

10.3 R code එක, පේළියෙන් පේළිය

```
# PCA cat example
setwd("C:\\Users\\samee\\My Drive\\Teaching\\ST 4035 - Data Science\\2024\\PCA")
library(jpeg)
cat <- readJPEG('cat.jpeg')
ncol(cat)       # columns 600
nrow(cat)       # පේළි 398
dim(cat)        # 398 600 3  -> 3-D array එකක්
```

- `setwd()` working directory එක (image එක තියෙන folder එක) සකසනවා. මේ path එක **ඔයාගේම folder එකට** වෙනස් කරන්න.

- `library(jpeg)` jpeg package එක load කරනවා (පළමු වතාවට `install.packages("jpeg")`).
- `readJPEG()` image එක අංක බවට හරවනවා (smiley face එක වගේ). ප්‍රතිඵලය **3-D array** එකක්: පේළි 398 × columns 600 × layers 3 (R, G, B). අගයන් 0 ත් 1 ත් අතර.

```
r <- cat[,,1]      # red channel   (398 x 600 matrix)
g <- cat[,,2]      # green channel
b <- cat[,,3]      # blue channel
dim(r)

cat.r.pca <- prcomp(r, center = FALSE)
cat.g.pca <- prcomp(g, center = FALSE)
cat.b.pca <- prcomp(b, center = FALSE)

rgb.pca <- list(cat.r.pca, cat.g.pca, cat.b.pca)
```

- `cat[,,1]` = ඔක්කොම පේළි, ඔක්කොම columns, layer 1 → රතු matrix එක. කොළ සහ නිල් ඒ විදිහටම.
- PCA කරන්නේ **හැම channel එකකටම වෙන වෙනම**. මෙතන data **standardize කරලා නැහැ** (`center = FALSE`) – මේ උදාහරණය පොතකින් ගත්තේ. හැම channel එකකම columns (variables) 600ක්.
- ප්‍රතිඵල තුන එක `list` එකකට දානවා.

```
# first 3
for (i in seq.int(3, round(nrow(cat) - 10), length.out = 10)) {
  pca.img <- sapply(rgb.pca, function(j) {
    compressed.img <- j$x[,1:i] %*% t(j$rotation[,1:i])
  }, simplify = 'array')
  writeJPEG(pca.img, paste('cat_compressed_', round(i,0), '_components.jpg', sep = ''))
}
```

- `seq.int(3, 388, length.out = 10)` 3 සිට 388 දක්වා අංක 10ක් දෙනවා: **3, 46, 89, 131, 174, 217, 260, 302, 345, 388**. මේවා තියාගන්න PCs ගණන්.
- හැම i එකකටම සහ හැම channel j එකකටම: `j$x[,1:i] %*% t(j$rotation[,1:i])` = (පළමු PC scores i) × (පළමු loadings i හි transpose එක). මේකෙන් PCs i ක් විතරක් භාවිතා කරලා channel එක **නැවත හදනවා**.
- `sapply(..., simplify = 'array')` නැවත හැදූ channels 3 ආයෙත් 3-D පාට image එකකට දානවා.
- `writeJPEG()` ඒක save කරනවා, උදා: `cat_compressed_46_components.jpg`.

```
original <- file.info('cat.jpeg')$size / 1000     # ප්‍රමාණය KB වලින්
imgs <- dir('compressed/')

for (i in imgs) {
  full.path <- paste('compressed/', i, sep='')
  print(paste(i, ' size: ', file.info(full.path)$size / 1000,
              ' original: ', original, ' % diff: ',
              round((file.info(full.path)$size / 1000 - original) / original, 2) * 100,
              '%', sep = ''))
}
```

- මේකෙන් compress කළ හැම file එකකම ප්‍රමාණය මුල් එක එක්ක (KB වලින්) සසඳලා % වෙනස print කරනවා.

- පන්තියේදී: PCs 3 → ≈ **16.7 KB**, තව PCs → ≈ 28.4 KB, PCs 388 → ≈ **30 KB**; මුල් එක ≈ **50 KB**. PCs වැඩි → file එක ලොකුයි, පින්තූරය පැහැදිලියි.

> **මේ code එක run කරන්න ප්‍රායෝගික tips**
>
> - Loop එක files save කරන්නේ working directory එකට, නමුත් size check එක කියවන්නේ `compressed/` කියන folder එකෙන්. Files ඒ folder එකට දාන්න, නැත්නම් කෙලින්ම එතනට save කරන්න: `paste('compressed/cat_compressed_', ...)` (මුලින් folder එක හදන්න).
> - නැවත හැදූ අගයන් 0 ට ටිකක් අඩු හෝ 1 ට වැඩි වෙන්න පුළුවන්. `writeJPEG` error දුන්නොත් clip කරන්න: `pca.img[pca.img < 0] <- 0; pca.img[pca.img > 1] <- 1.`

> **ශිෂ්‍ය ප්‍රශ්නය: "ප්‍රතිඵල image එකේ resolution එක අඩු වෙන්නේ නැත්තේ ඇයි?"**
> Pixels ගණන එහෙමමයි (398 × 600), නමුත් **තොරතුරු ප්‍රමාණය** අඩු වෙනවා. වෙනස පේනවා: PCs ටිකක් තියෙනකොට පූසා පේන්නේ නැහැ, PCs වැඩි කරද්දී පැහැදිලියි නමුත් noise තියෙනවා, file sizes වෙනස් (16.7 KB, 28.4 KB, 30 KB, 50 KB එදිරිව).

11. PCA හි න්‍යාය – රූපමය පැහැදිලි කිරීම

මේ slides හදලා තියෙන්නේ ගණිතය නොකරන (ජීව විද්‍යා / bioinformatics) ශිෂ්‍යයන්ට, PCA ගැන **StatQuest** YouTube video එක මත පදනම්ව. ගණිතය රූප වලින් පැහැදිලි කරනවා.

11.1 Data

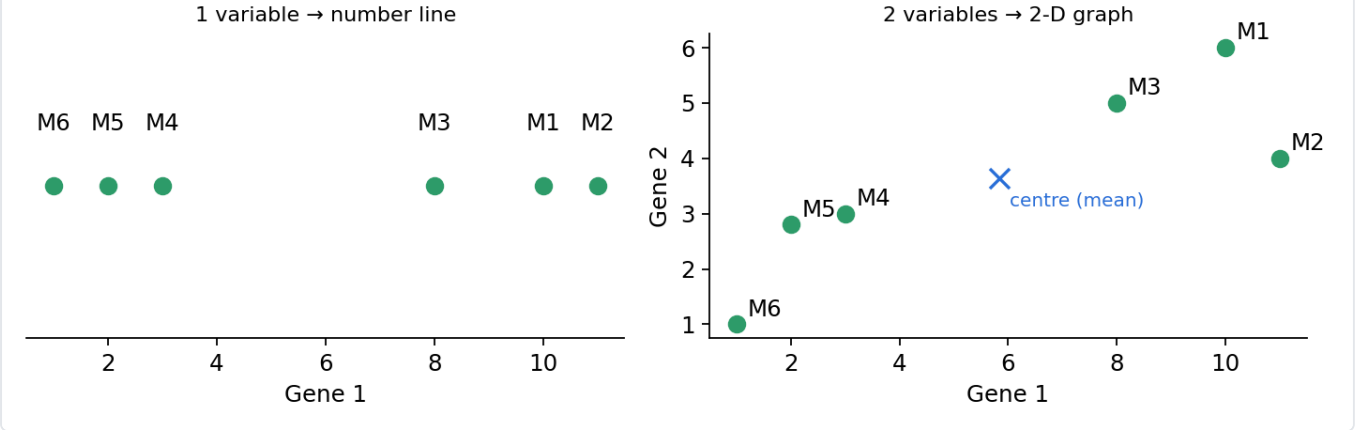
	Mouse 1	Mouse 2	Mouse 3	Mouse 4	Mouse 5	Mouse 6
Gene 1	10	11	8	3	2	1
Gene 2	6	4	5	3	2.8	1

Let's start with a simple data set.

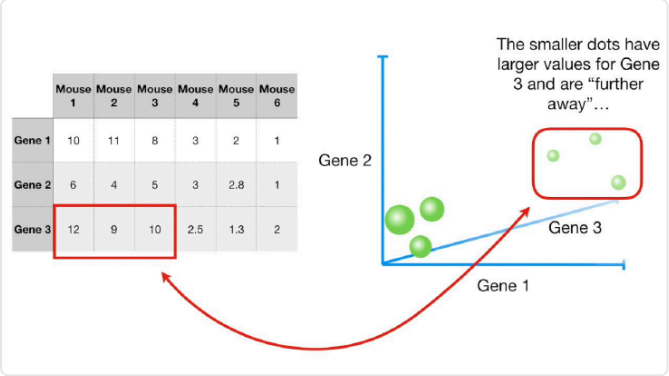
මීයන් 6 දෙනෙකුගේ genes 2ක gene expression

- Bioinformatics වලදී **gene expression levels** මනිනවා (උස හෝ බර මනිනවා වගේ). මෙතන මීයන් 6 දෙනෙකුගේ genes 2ක් මැන්නා.
- සටහන: මෙතන **variables පේළි වල**, **samples (මීයෝ) columns වල** – bioinformatics වල මේක සාමාන්‍යයි.
- තවත් උදාහරණයක්: samples උසස් පාසල් ශිෂ්‍යයෝ, variables test ලකුණු (උදා: ශිෂ්‍ය 1: ගණිතය 95, කියවීම 96; ශිෂ්‍ය 2: ගණිතය 88, කියවීම 79 ...).

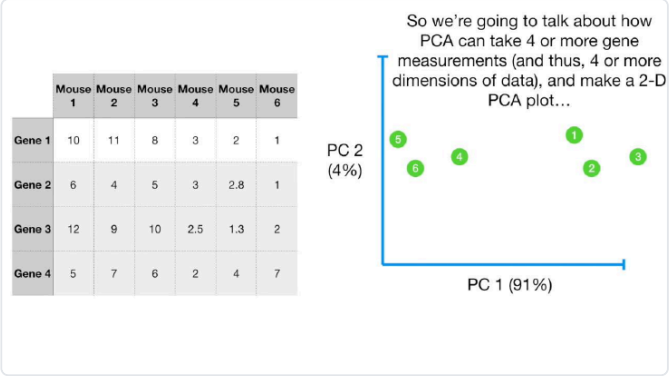
11.2 එකක්, දෙකක්, තුනක් ... variables ගොඩක්



- **Gene 1ක් → අංක රේඛාවක්** මත plot කරනවා. ඉහළ සහ පහළ අගයන් ලේසියෙන් පේනවා.
- **Genes 2ක් → 2-D graph** එකක් (x මත Gene 1, y මත Gene 2). රටා පේනවා: මීයන් 1, 2, 3 එකට, 4, 5, 6 එකට.
- **Genes 3ක්** → 3-D graph එකක් (නැත්නම් 3 වන variable එක bubble එකේ **ප්‍රමාණයෙන්** පෙන්නනවා).
- **Genes 4ක් හෝ වැඩි** → visualize කරන්න **බැහැ**. Data සාරාංශ කරන්න ඕන → PCA මගින් **dimensions අඩු කිරීම**.



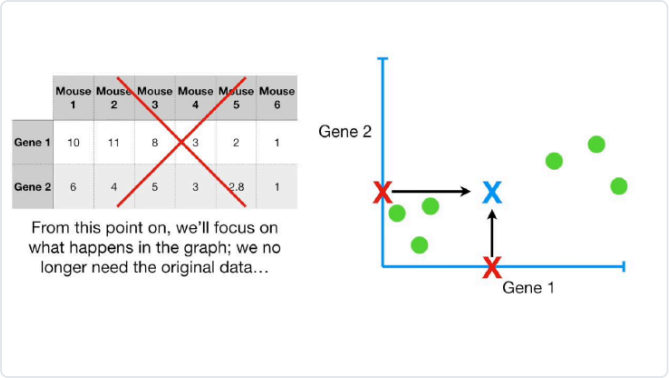
3 වන gene එක අලුත් අක්ෂයක් ලෙස



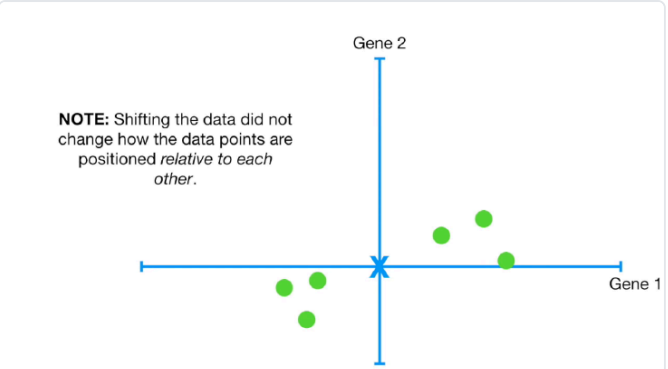
PCA න් පස්සේ: 2-D PCA plot එකක්

PCA න් පස්සේ PC1 සහ PC2 plot කරනවා. උදාහරණයක් විදිහට PC1 91% ක් සහ PC2 4% ක් පැහැදිලි කරනවා → එකට 95%, ඒක ප්‍රමාණවත්. Clusters වෙන් කරන්න **වැඩිපුරම වගකිව යුතු variable එක** (උදා: Gene 3) කුමක්ද කියලත් PCA එකෙන් කියන්න පුළුවන්.

11.3 පියවර 1 – Data මධ්‍යගත (centre) කිරීම



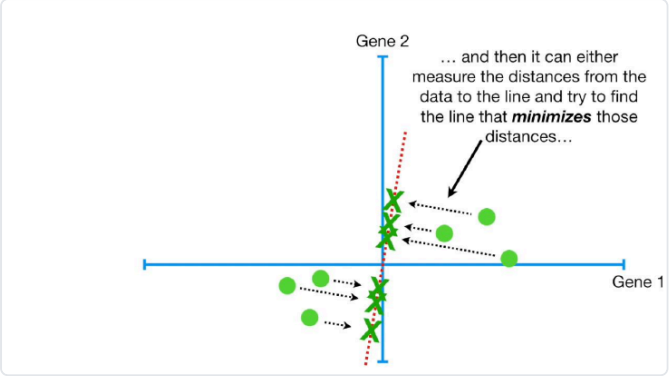
Gene 1 සහ Gene 2 සාමාන්‍යයෙන් මධ්‍යය ලැබෙනවා



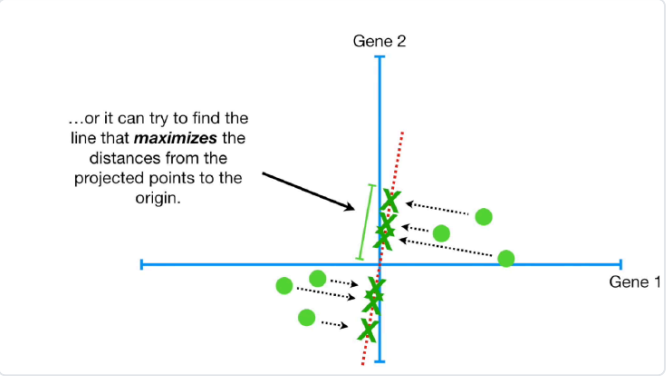
මධ්‍යය මූලය (origin) මත වෙන්න data මාරු කරනවා

- **හැම variable එකකම mean එක** සොයන්න (points හැම අක්ෂයකටම project කරලා සාමාන්‍යය ගන්න). ඉහත data වලින්: Gene 1 mean = 35/6 = 5.83, Gene 2 mean = 21.8/6 = 3.63.
- (Gene 1 mean, Gene 2 mean) කියන point එක data වල **මධ්‍යය**.
- මධ්‍යය **origin (0, 0)** මත වෙන්න data **මාරු (shift)** කරන්න. ප්‍රායෝගිකව: හැම data point එකකින්ම **mean එක අඩු කරන්න** (උදා: මීයා 1: 10 − 5.83 = 4.17, 6 − 3.63 = 2.37).
- මාරු කිරීමෙන් points එකිනෙකට සාපේක්ෂව තියෙන විදිහ **වෙනස් වෙන්නේ නැහැ** – එකම පරතර තියෙනවා. වෙනස් වෙන්නේ origin එක විතරයි.

11.4 පියවර 2 – Origin හරහා හොඳම රේඛාව fit කිරීම



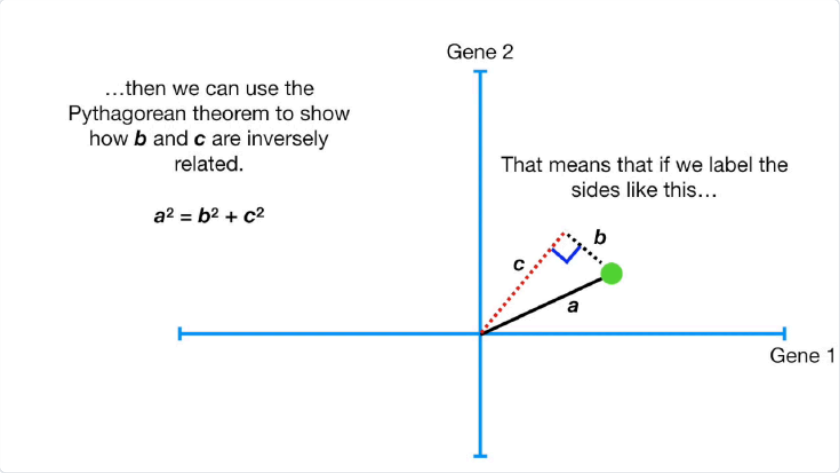
Points සිට රේඛාවට දුර අවම කරන්න…



…නැත්නම් projected points සිට origin එකට දුර උපරිම කරන්න

- Origin හරහා අහඹු රේඛාවක් අඳින්න, ඊට පස්සේ data වලට හොඳින්ම ගැලපෙනකම් ඒක **කරකවන්න (rotate)**.
- "හොඳම" රේඛාවේ points සිට රේඛාවට ඇති **අපගමනය (deviations) අඩුමයි**.
- **Projection:** හැම point එකක්ම සෘජු කෝණයකින් රේඛාවට බස්සන්න. ඒක වැටෙන තැන projection එක.
- රේඛාව හොඳින් ගැලපෙද්දී, හැම point එකකම සිට රේඛාවට දුර **අඩු වෙනවා**, projected point එකේ සිට origin එකට දුර **වැඩි වෙනවා**.

11.5 ඇයි? – පයිතගරස් ප්‍රමේයය

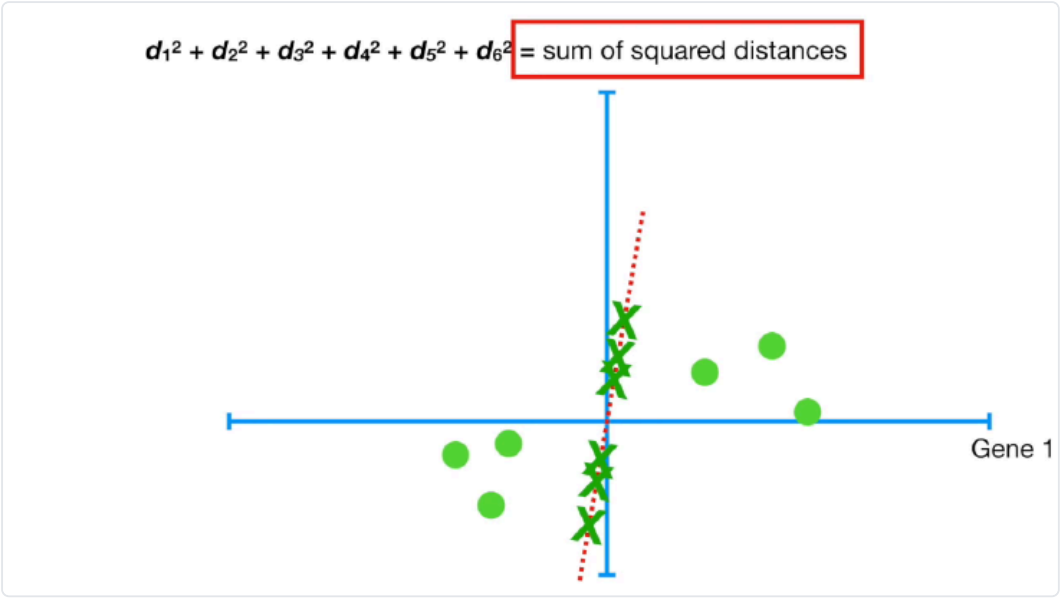


එක point එකක් බලන්න. Origin සිට point එකට දුර (a) රේඛාව කරකවද්දී වෙනස් **වෙන්නේ නැහැ**. Projection එක **සෘජු කෝණයක්** හදනවා, ඒ නිසා පයිතගරස් අනුව:

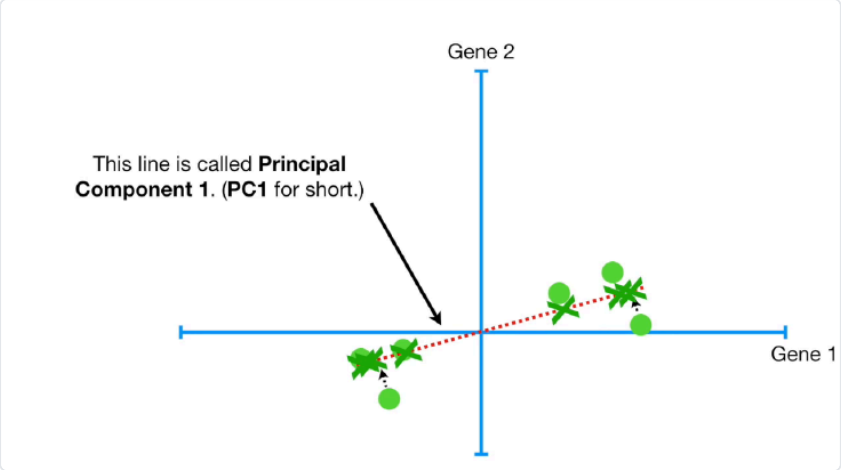
$$a^2 = b^2 + c^2$$

මෙහි b = projected point සිට origin එකට දුර, c = point සිට රේඛාවට දුර. a^2 නියත නිසා: **b ලොකු වෙනවා නම් c කුඩා විය යුතුයි**, අනිත් පැත්තටත් එහෙමයි. ඒ නිසා PCA එකට **c අවම කරන්න** (රේඛාවට දුර) හෝ **b උපරිම කරන්න** (projected point සිට origin එකට දුර) පුළුවන්. දෙකම එකම දේ. ගණනය කරන්න ලේසි නිසා PCA b උපරිම කරනවා.

11.6 වර්ග කළ දුරවල එකතුව (Sum of squared distances)

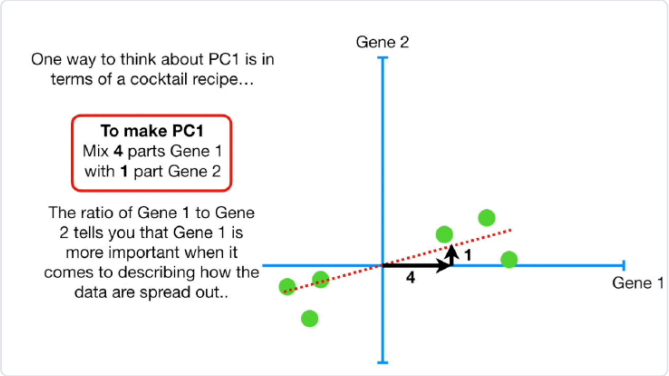


- හැම projected point එකකම සිට origin එකට දුර මනින්න: d1, d2, ..., d6.
- සමහර ඒවා ධන පැත්තේ, සමහර ඒවා ඍණ පැත්තේ, ඒ නිසා එකිනෙක කැපී යනවා. ඍණ අගයන් ධන අගයන් කපා නොදමන්න ඒවා **වර්ග කරනවා**.
- එකතු කරන්න: $d1^2 + d2^2 + \ldots + d6^2$ = **වර්ග කළ දුරවල එකතුව, SS(distances)**.
- රේඛාව කරකවන්න, project කරන්න, SS ආයෙත් ගණනය කරන්න ... **විශාලතම SS(distances)** ලැබෙනකම්.

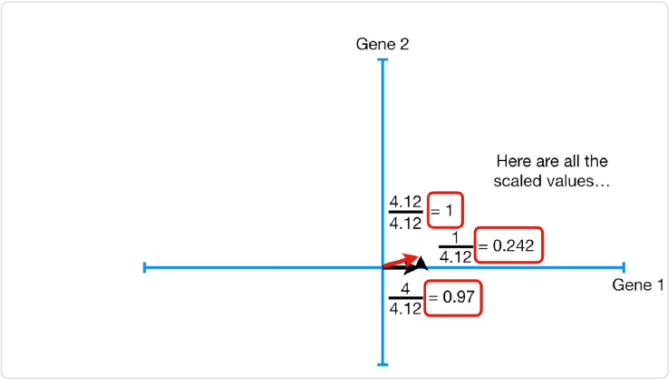


විශාලතම SS(distances) ඇති රේඛාව Principal Component 1 (PC1)

11.7 PC1 වට්ටෝරුවක් ලෙස – loading scores

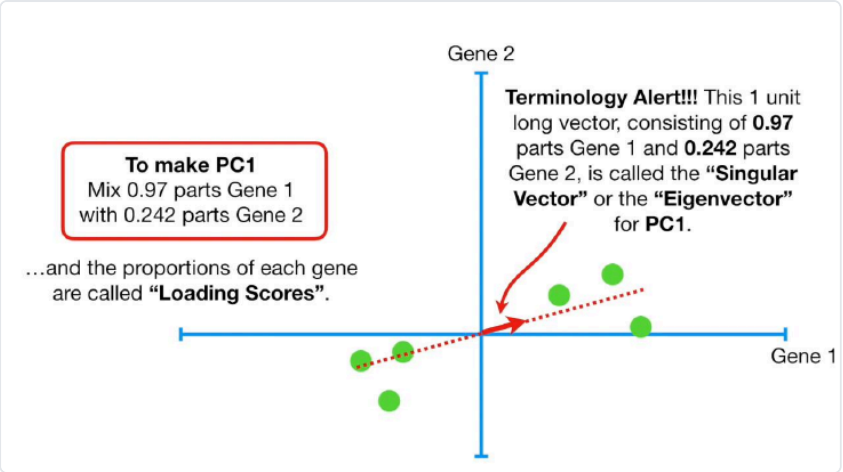


Slope 0.25: Gene 1 දිගේ 4ක් යන්න, Gene 2 දිගේ 1ක් ඉහළට



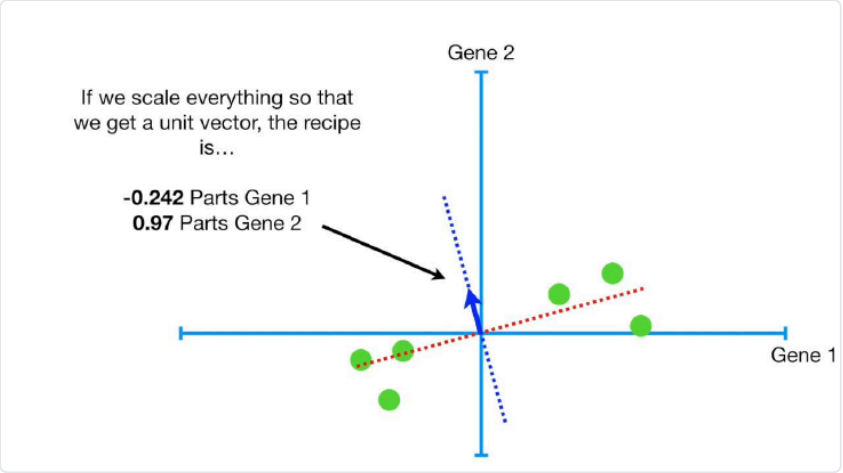
ඒකක දෛශිකයක් (unit vector) කරන්න: 4.12 න් බෙදන්න

- Origin හරහා යන රේඛාවක් නිර්වචනය වෙන්නේ එහි **slope (අනුක්‍රමණය)** එකෙන් විතරයි (intercept එක 0). Slide උදාහරණයේ PC1 හි **slope එක 0.25**: **Gene 1 දිගේ ඒකක 4 කට**, Gene 2 දිගේ **ඒකක 1ක් ඉහළට** (1/4 = 0.25 = tan θ).
- ඒ කියන්නේ data වැඩිපුරම පැතිරිලා තියෙන්නේ Gene 1 දිගේ, Gene 2 දිගේ පොඩ්ඩයි.
- **Cocktail වට්ටෝරුව:** "PC1 හදන්න **Gene 1 කොටස් 4ක් Gene 2 කොටස් 1ක්** එක්ක මිශ්‍ර කරන්න." Data පැතිරෙන විදිහ විස්තර කරන්න Gene 1 වඩා වැදගත්.
- ගණිතමය වශයෙන්, PC1 කියන්නේ Gene 1 සහ Gene 2 හි **linear combination** එකක්.
- **ඒකක දෛශිකයක් කරන්න** (දිග 1): දිගම පැත්ත = $\sqrt{(4^2 + 1^2)} = \sqrt{17} = \mathbf{4.12}$. හැම පැත්තක්ම 4.12 න් බෙදන්න: 4/4.12 = **0.97**, 1/4.12 = **0.242**, 4.12/4.12 = 1.
- අලුත් වට්ටෝරුව: **Gene 1 කොටස් 0.97 + Gene 2 කොටස් 0.242**. එකම දිශාව, standardize කළා විතරයි.



වචනය	තේරුම
PC1 සඳහා **Eigenvector** (singular vector)	ඒකක දෛශිකය (0.97, 0.242)
Loading scores	0.97 සහ 0.242 අංක (හැම variable එකකම අනුපාත). R වල: `$rotation`
PC1 සඳහා **Eigenvalue**	PC1 සඳහා SS(distances)
PC1 සඳහා **Singular value**	√(PC1 සඳහා eigenvalue)

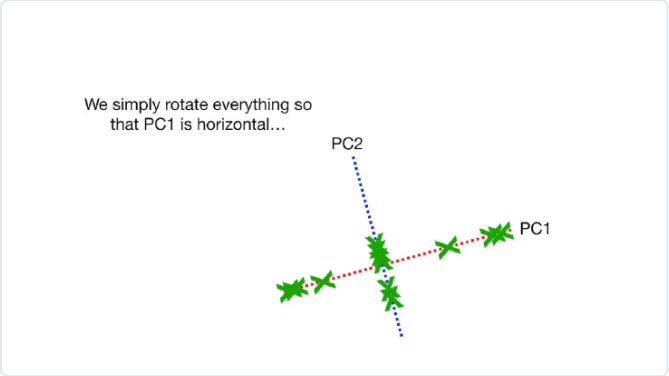
11.8 PC2



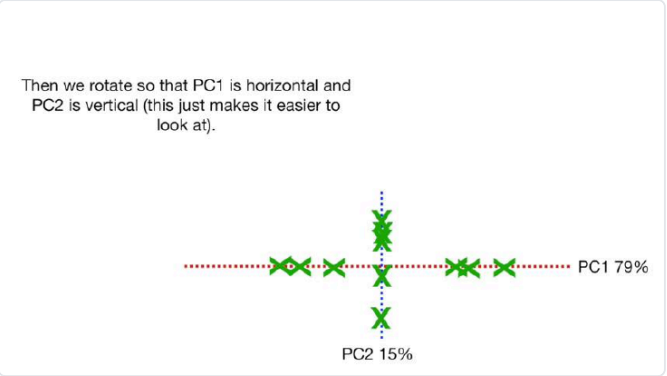
- PCs එකිනෙකට **orthogonal** = **ලම්බක (perpendicular)** (සංඛ්‍යානයේදී: **ස්වාධීන** / correlation නැති).
- ඉහළ dimensions වල PC1 ට ලම්බක රේඛා **අනන්ත ගණනක්** තියෙනවා (PC1 වටේ කරකවන්න පුළුවන්). PC2 කියන්නේ **දෙවනියට විශාලතම SS(distances)** ඇති ලම්බක රේඛාව.
- Variables 2ක් විතරක් තියෙනකොට PC2 කියන්නේ සරලවම origin හරහා PC1 ට ලම්බක රේඛාව – තවත් optimization ඕන නැහැ.
- PC2 වට්ටෝරුව: **Gene 1 කොටස් −1 + Gene 2 කොටස් 4**. ඒකක දෛශිකයක් ලෙස: **−0.242 Gene 1 + 0.97 Gene 2**. මේවා PC2 සඳහා loading scores (PC2 සඳහා Gene 2, Gene 1 ට වඩා 4 ගුණයක් වැදගත්).

- PC2 සඳහා eigenvalue = PC2 සඳහා SS(distances).

11.9 අවසාන PCA plot එක



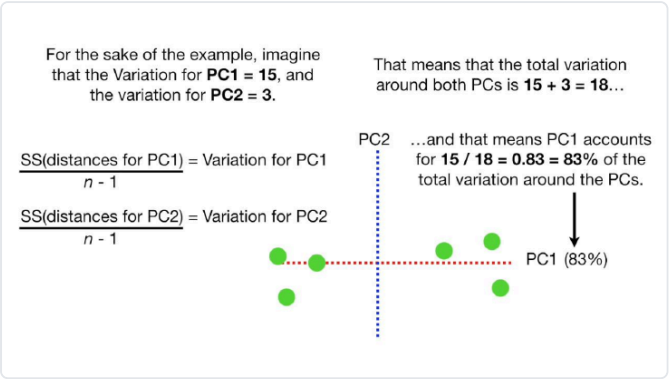
PC1 තිරස් වෙන්න කරකවන්න

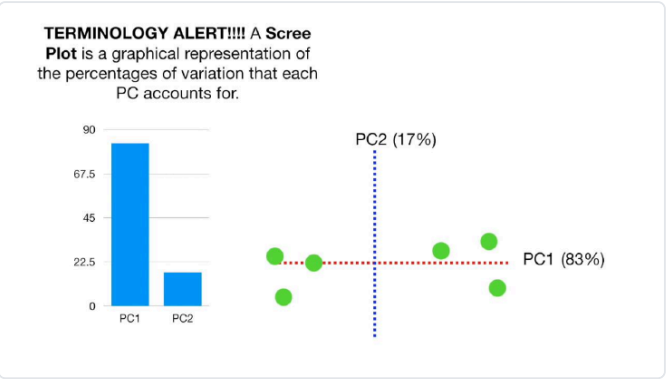


හැම sample එකක්ම තැබීමට projected points භාවිතා කරන්න

PC1 x-අක්ෂය සහ **PC2 y-අක්ෂය** වෙන්න ඔක්කොම කරකවන්න. හැම sample එකක්ම PC1 සහ PC2 මත එහි projected points භාවිතා කරලා තබනවා. මේවා අලුත් transform කළ variables.

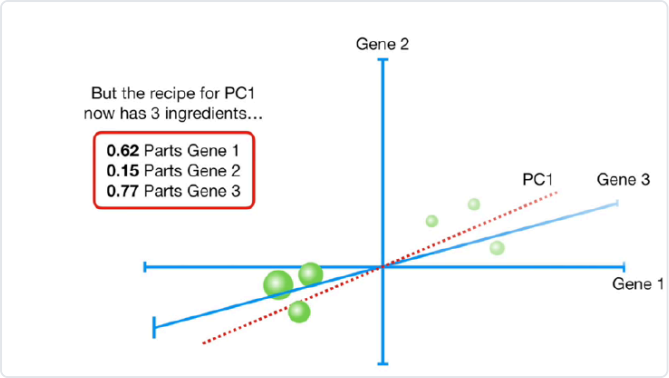
11.10 Variation explained සහ scree plot



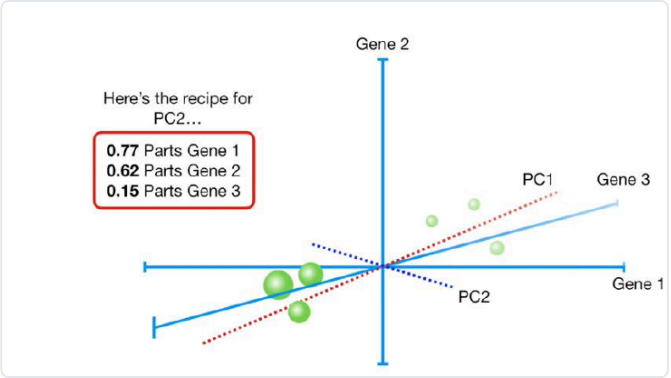


- Sample size සෘණ 1 න් බෙදලා SS(distances) **විචලනය (variation)** බවට හරවන්න: **Variation = SS(distances) / (n − 1)**. මේක sample variance සූත්‍රය $\Sigma(x_i - \bar{x})^2 / (n - 1)$ ම තමයි.
- උදාහරණය: PC1 variation = 15, PC2 variation = 3 → මුළු එකතුව = 18.
- PC1 පැහැදිලි කරන්නේ 15/18 = **83%**, PC2 පැහැදිලි කරන්නේ 3/18 = **17%**.
- **Scree plot** එකක් කියන්නේ හැම PC එකක්ම පැහැදිලි කරන විචලන ප්‍රතිශතයන්ගේ graph එකක්.

12. Variables 3ක් හෝ වැඩි විට PCA, සහ PCA වැඩ නොකරන අවස්ථා

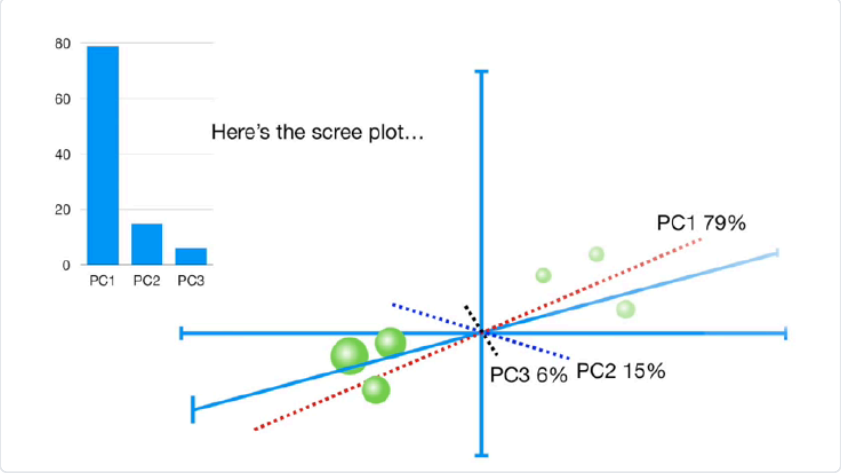


Genes 3ක් ඇති විට PC1 වට්ටෝරුව



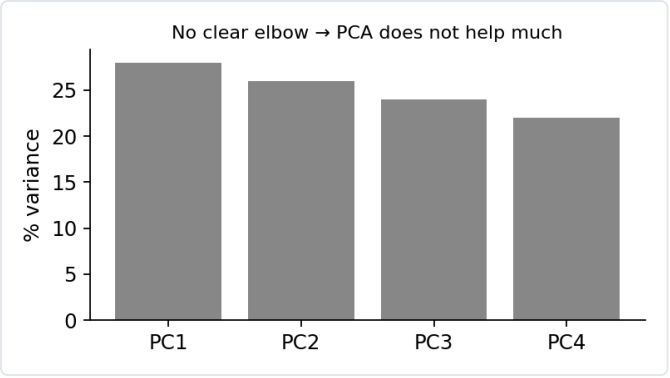
PC2 වට්ටෝරුව

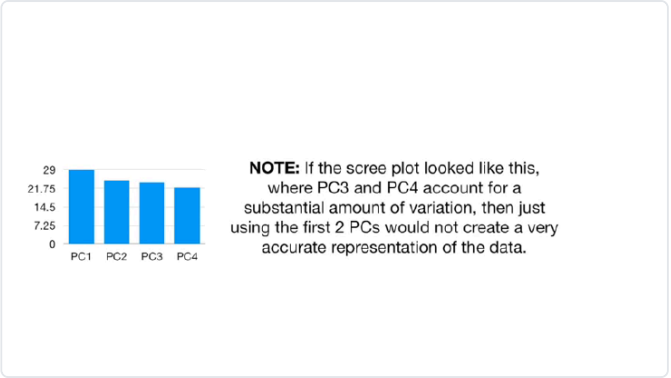
- Genes 3ක් තියෙනකොටත් එකම දේ කරනවා: data centre කරන්න, origin හරහා හොඳින්ම ගැලපෙන රේඛාව (PC1) සොයන්න, ඊට පස්සේ PC1 ට ලම්බකව PC2, ඊට පස්සේ දෙකටම ලම්බකව PC3.
- PC1 වට්ටෝරුව: 0.62 Gene 1 + 0.15 Gene 2 + **0.77 Gene 3** → PC1 සඳහා වඩාත්ම වැදගත් Gene 3.
- PC2 වට්ටෝරුව: **0.77 Gene 1** + 0.62 Gene 2 + 0.15 Gene 3 → PC2 සඳහා වඩාත්ම වැදගත් Gene 1.
- න්‍යායාත්මකව, variable එකකට PC එකක් තියෙනවා (නැත්නම් sample එකකට, අඩු එක).
- මෙතන PC1 = 79%, PC2 = 15%, PC3 = 6%. ඒ නිසා PC1 + PC2 = **94%**. PC1 සහ PC2 හි 2-D plot එකක් 3-D graph එකට හොඳ ආසන්නයක්. අපිට නැති වෙන්නේ 6% ක් විතරයි, නමුත් සරල 2-D පින්තූරයක් ලැබෙනවා – සාධාරණ හුවමාරුවක්.



- **Genes 4ක්** තියෙනකොට 4-D graph එකක් අඳින්න බැහැ, නමුත් PCA තවමත් වැඩ කරනවා: උදා: PC1 + PC2 90% ක් පැහැදිලි කරනවා → ඒවා 2-D වල plot කරන්න.

PCA උදව් නොවන විට





Scree plot එකේ **පැහැදිලි elbow එකක් නැත්නම්** – ඔක්කොම PCs වල උස දළ වශයෙන් සමානයි නම් (උදා: PC3 සහ PC4 ත් විශාල ප්‍රමාණයක් පැහැදිලි කරනවා) – පළමු PCs දෙක විතරක් භාවිතා කිරීමෙන් නිවැරදි පින්තූරයක් **ලැබෙන්නේ නැහැ**. PCs අතහැරියොත් **තොරතුරු නැතිවීම සැලකිය යුතුයි**. කෙසේ වෙතත්, noise තියෙන PCA plot එකක් වුණත් clusters හඳුනාගන්න භාවිතා කරන්න පුළුවන්.

> **ලබන සතියේ**
> **ඔක්කොම** variables **categorical** නම් මොකද වෙන්නේ? එතකොට PCA භාවිතා කරන්න බැහැ (ඔක්කොම ඉවත් කරන්න වෙනවා). ලබන සතියේ: එවැනි data සඳහා **වෙනත් dimension reduction techniques**.

මූලාශ්‍රය: StatQuest, "Principal Component Analysis (PCA), Step-by-Step" – **youtu.be/FgakZw6K1QQ**. StatQuest එකේ බොහෝ statistics සහ data science මාතෘකා ගැන ඉතා පැහැදිලි විස්තර තියෙනවා.

13. සාරාංශය, ගෙදර වැඩ සහ පුනරීක්ෂණ ප්‍රශ්න

ප්‍රධාන කරුණු

- Dimension = variables ගණන. Dimension reduction මගින් තොරතුරු වැඩි ප්‍රමාණයක් රඳවාගන්නා **අලුත් variables කුඩා කට්ටලයක්** හදනවා.
- PCs කියන්නේ ඔක්කොම මුල් variables වල **linear combinations**; PC1 වැඩිම විචලනය, PC2 ඊළඟට, ආදී වශයෙන් පැහැදිලි කරනවා. PCs **orthogonal** (correlation නැති).
- PCA: **quantitative data විතරයි** → **standardize** → covariance matrix → eigenvalues/eigenvectors → loadings → transform කළ data → PCs ගණන තෝරන්න.
- PCs තෝරන්නේ **scree (elbow) plot** සහ **cumulative % variance** එකෙන් (ක්ෂේත්‍රය අනුව 80%–95%).
- R: `prcomp(x, center = TRUE, scale. = TRUE)`, `summary()`, `plot()` / `screeplot()`, `biplot()`, `$rotation` = loadings, `$x` = scores.
- mtcars: vs සහ am ඉවත් කරලා standardize කළාම → **PCs 3ක් 91.58%** ක් පැහැදිලි කරනවා.
- ජ්‍යාමිතිය: data centre කරන්න, projected points වල **SS(distances) උපරිම කරන** රේඛාව සොයන්න (පයිතගරස් අනුව = රේඛාවට දුර අවම කිරීම). ඒකක දෛශිකය = eigenvector; SS = eigenvalue; SS/(n−1) = variance.
- PCA යනු **අතරමැදි පියවරක්** (ඊට පස්සේ regression, clustering…). යෙදීමක්: **image compression**.

> **ගෙදර වැඩ**
> mtcars සඳහා PC2 සහ PC3 ලියන්න (උත්තර 8 වන කොටසේ – ඔයාගේම R output එක එක්ක check කරන්න).

ඔයාම පරීක්ෂා කරන්න

1. Data set එකක "dimension" කියන්නේ මොකක්ද? Dimension reduction variables මකනවද?
2. Variables 100ක් සඳහා පළමු principal component එකේ පොදු ආකාරය ලියන්න.
3. PCA හි පියවර ලැයිස්තුගත කරන්න.
4. PCA ට කලින් standardize කළ යුත්තේ ඇයි? mtcars වල එහෙම නොකළොත් මොකද වෙන්නේ?
5. vs සහ am ඉවත් කරන්න `sapply(data, is.numeric)` භාවිතා කරන්න බැරි ඇයි?
6. PCA කරන R functions දෙක මොනවද, ඒවා තියෙන්නේ කුමන package එකේද?
7. PC1 = 50%, PC2 = 30%, PC3 = 12%, PC4 = 5%. පළමු PCs 3 සඳහා cumulative % සොයන්න. (උත්තරය: 92%)
8. Elbow point එකක් කියන්නේ මොකක්ද? Scree plot එකේ elbow එකක් නැත්නම් එහි තේරුම මොකක්ද?
9. `prcomp` output එකේ loadings සහ transform කළ data තියෙන්නේ කොහේද?
10. PC1 හි slope එක 0.25. ඒකක දෛශික loading scores සොයන්න. (උත්තරය: 0.97 සහ 0.242)
11. රේඛාවට දුර අවම කිරීම, projected points සිට origin එකට දුර උපරිම කිරීමට සමාන වන්නේ ඇයිදැයි පයිතගරස් භාවිතයෙන් පැහැදිලි කරන්න.
12. PC1 variation = 12, PC2 = 4. එක එකක් පැහැදිලි කරන්නේ කොපමණ % ද? (උත්තරය: 75% සහ 25%)
13. Image එකක් compress කරන්න PCA භාවිතා කරන්නේ කොහොමදැයි පැහැදිලි කරන්න.